Project Report
on
Student Management System

Object Oriented Programming

Computer Science and Engineering,

Techno Main Salt Lake

Prepared by:

Section Group: A1

Team No: 2

Prateek Kalwar (13000124018)

Chandrima Saha (13000123031)

Debadrita Banerjee (13000125235)

Pratush Raj (13000124019)

Ayush Kumar Singh (13000124007)

Table of Contents

1. Problem statement....1
2. Solution Design....1
    2.1. Class Diagram....1
    2.2. Database Schema....2
    2.3. GUI Component Design & Layout....4
    2.4. Logic & Event Handling....6
    2.5. Inputs, Outputs and Error handling....8
3. Test results....9
4. Conclusion....10
5. References....11

1. Problem statement

Educational institutions have to keep track of their students, the courses they offer, and how each student is performing in every course. When these details are kept on paper or in separate spreadsheets, the same data is typed more than once, individual records are slow to find and correct, marks and grades are worked out inconsistently, and students whose attendance is falling are noticed late.

The aim of this mini project is to design and build a desktop application in Java, called the **Student Management System**, that:

- stores students (roll number, name, email, department, phone), courses (course code, name, credits, instructor) and enrollments in a relational SQLite database;
- lets the user create, view, search, update and delete student and course records through a Swing graphical interface;
- enrolls a student in a course and records attendance percentage and exam marks, then computes the total score and grade automatically, where the total of 100 is made up of 90 exam marks and up to 10 attendance marks;
- shows a dashboard with summary figures and a list of students whose attendance is below 75%;
- is organised using the Model-View-Controller (MVC) pattern so that the object-oriented design, GUI event handling and JDBC database connectivity taught in the course are all demonstrated.

2. Solution Design

The solution is a single desktop application made of four Java classes (Main, MainView, AppController and DatabaseModel) and one SQLite database file, student_management.db, which is created automatically in the folder from which the program is run. The sub-sections below give the class diagram, the database schema, the GUI design, the logic and event handling, and the input and error-handling rules, all taken from the source code of the project.

2.1. Class Diagram

Figure 1 shows the four classes of the project and their relationships. Main only starts the program and creates one object of each of the other three classes. MainView extends the Swing class JFrame and contains all visual components. AppController holds references to a MainView and a DatabaseModel (the fields view and model) and connects them. DatabaseModel is the only class that touches the database. Members that are public in MainView are the Swing components, which the controller needs in order to attach listeners and to read and write field values.

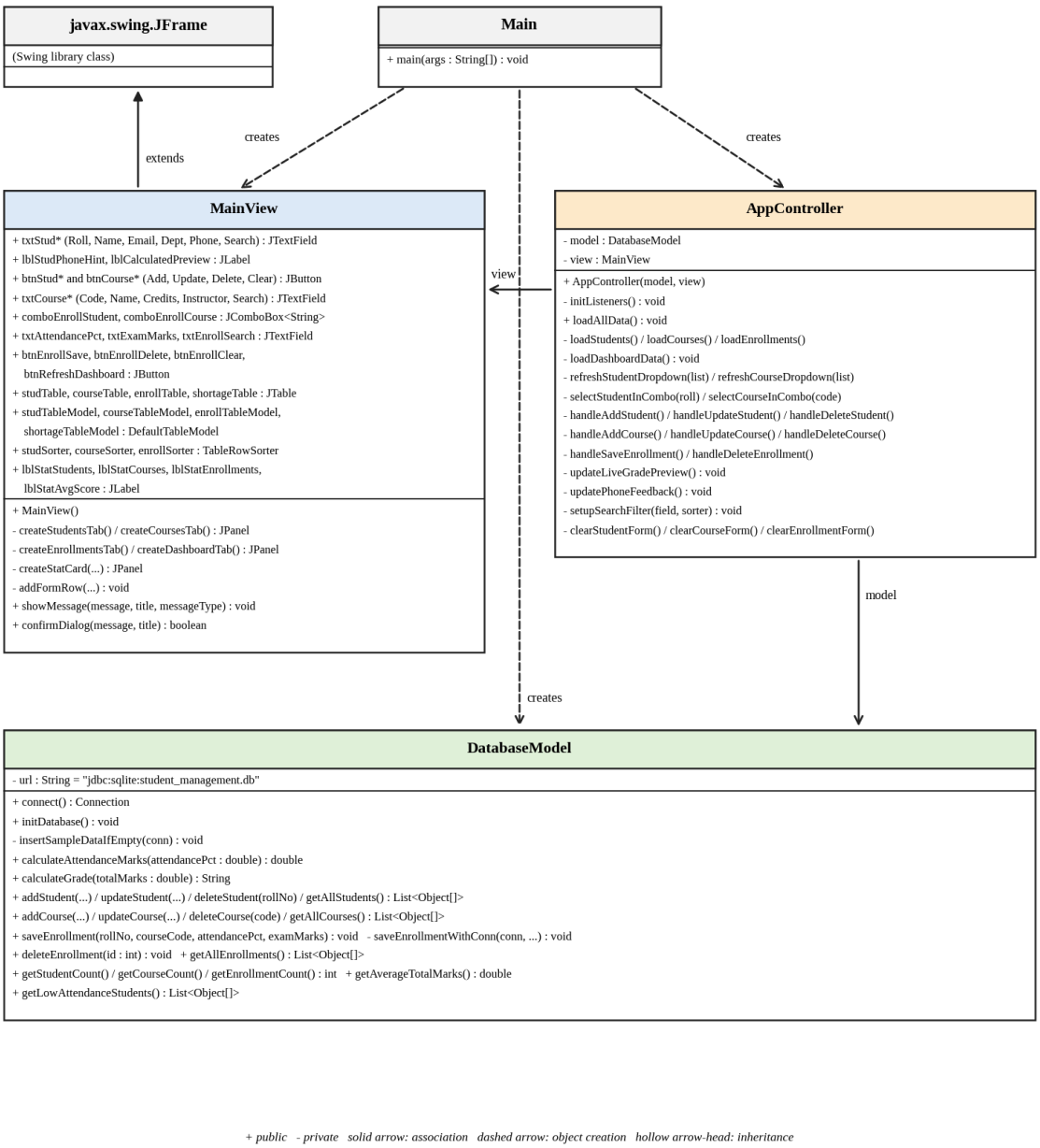
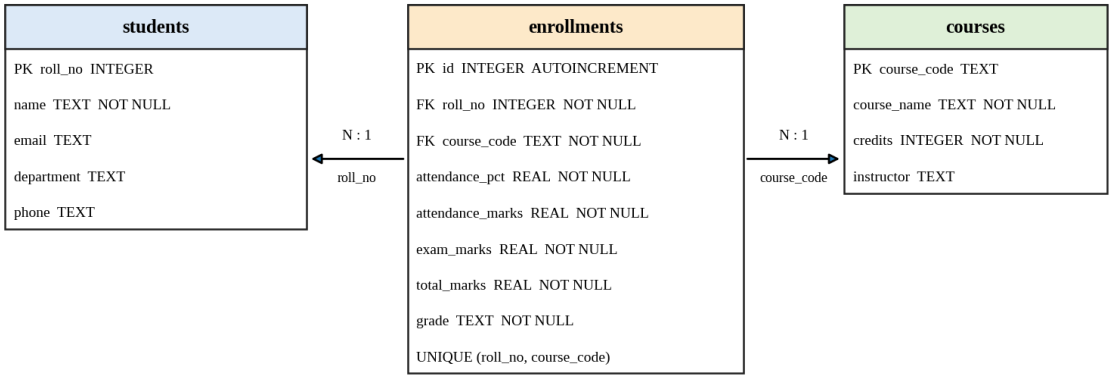


Figure 1: Class diagram of the Student Management System (members summarised where they repeat for students, courses and enrollments)

Besides these named classes, the code uses anonymous inner classes and lambda expressions: DefaultTableModel is subclassed anonymously (overriding isCellEditable to make the tables read-only), DocumentFilter is subclassed in AppController to restrict the phone field, and the interfaces ActionListener, ListSelectionListener and DocumentListener are implemented with lambdas or anonymous classes.

2.2. Database Schema

The database uses SQLite through JDBC (URL jdbc:sqlite:student_management.db). The method DatabaseModel.initDatabase() creates three tables with CREATE TABLE IF NOT EXISTS, so the schema is created the first time the program runs. If the students table is empty, insertSampleDataIfEmpty() adds three students (roll numbers 101 to 103), three courses (CS101 to CS103) and four sample enrollments. Figure 2 shows the relationships and Table 1 lists every column.



Arrows point from the foreign key to the referenced primary key (both with ON DELETE CASCADE).

Figure 2: Entity-relationship diagram of the database

Table 1: Database schema

Table	Column	Data type	Key / constraint
students	roll_no	INTEGER	PRIMARY KEY
	name	TEXT	NOT NULL
	email	TEXT	-
	department	TEXT	-
	phone	TEXT	-
courses	course_code	TEXT	PRIMARY KEY
	course_name	TEXT	NOT NULL
	credits	INTEGER	NOT NULL
	instructor	TEXT	-
enrollments	id	INTEGER	PRIMARY KEY AUTOINCREMENT
	roll_no	INTEGER	NOT NULL; FOREIGN KEY to students(roll_no) ON DELETE CASCADE
	course_code	TEXT	NOT NULL; FOREIGN KEY to courses(course_code) ON DELETE CASCADE
	attendance_pct	REAL	NOT NULL
	attendance_marks	REAL	NOT NULL
	exam_marks	REAL	NOT NULL
	total_marks	REAL	NOT NULL
	grade	TEXT	NOT NULL
	(roll_no, course_code)	-	UNIQUE (one enrollment per student per course)

A student can have many enrollments and a course can have many enrollments, so the enrollments table links the two tables through its foreign keys; this makes students and courses a many-to-many relationship. The UNIQUE constraint on (roll_no, course_code) together with the SQL clause INSERT ... ON CONFLICT(roll_no, course_code) DO UPDATE lets the same Save / Update Grade operation either create a new enrollment or update an existing one. Add, update and delete operations use PreparedStatement with ? placeholders, and the list and dashboard queries use JOINs of enrollments with students and courses.

Note on foreign keys. initDatabase() executes PRAGMA foreign_keys = ON on the connection it opens. In SQLite this setting applies to one connection only, and connect() opens a new connection for every later operation without repeating the pragma, so the ON DELETE CASCADE rules may not

be enforced when a student or course is deleted. This should be confirmed with test case xiv in Section 3.

2.3. GUI Component Design & Layout

The interface is built in the class MainView. A BorderLayout main panel holds a dark-blue header (two JLabels with the application name and the subtitle "MVC Architecture | Relational SQLite Database | Attendance-Weighted Grading") at the top and a JTabbedPane in the centre. The first three tabs share one layout: a JSplitPane with a titled form panel on the left and, on the right, a search bar above a JTable inside a JScrollPane. Clicking a table row loads it into the form. The fourth tab shows four KPI cards and the attendance shortage table. Figures 3 to 6 are screenshots of the running application.

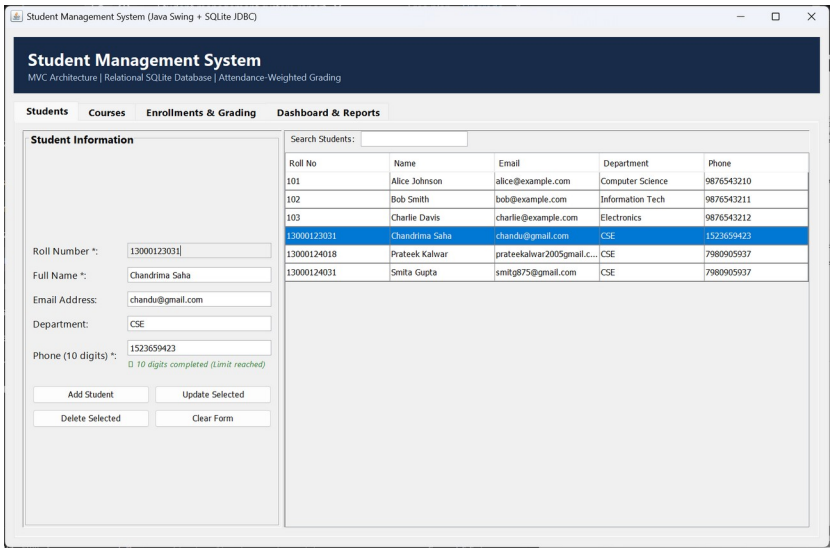


Figure 3: Students tab - form, Add / Update / Delete / Clear buttons, live search and student table

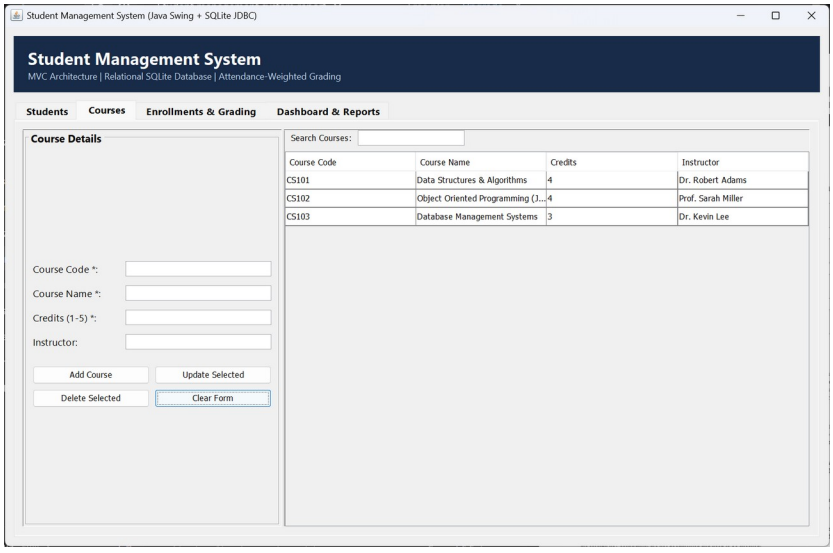


Figure 4: Courses tab - course form, buttons, search box and course table

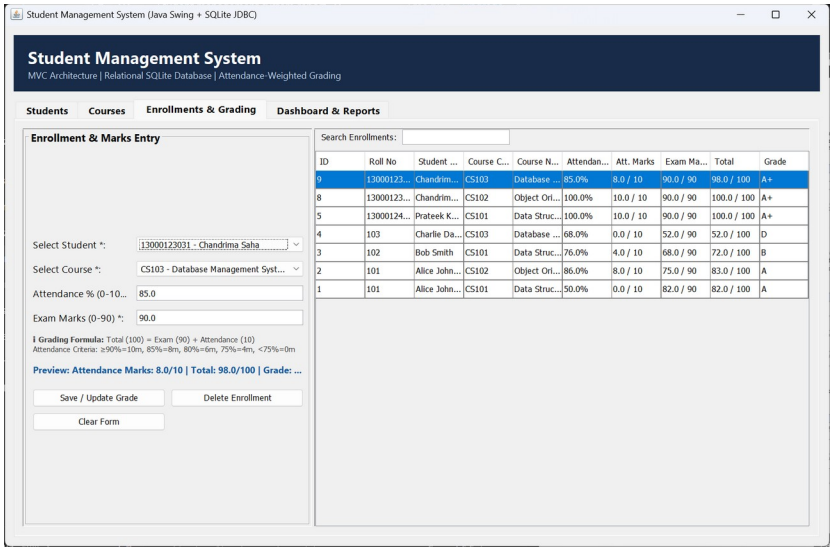


Figure 5: Enrollments & Grading tab - student and course drop-downs, marks fields, live preview and enrollment table

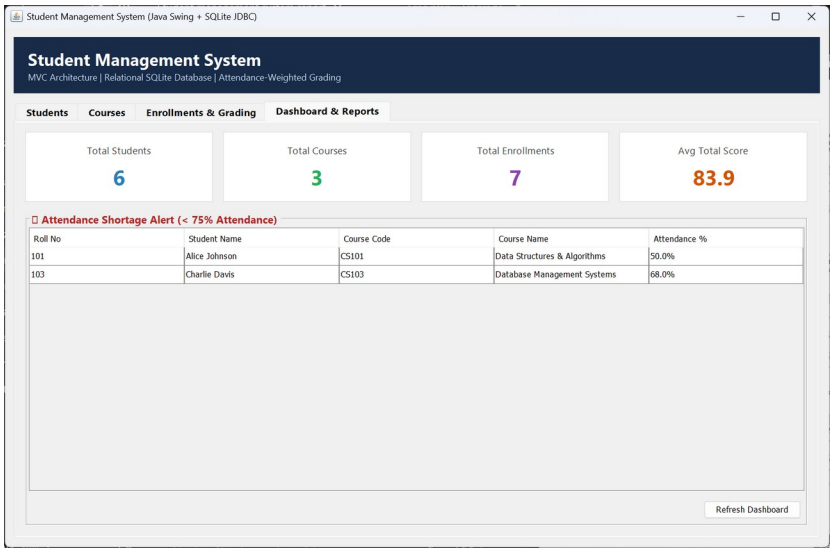


Figure 6: Dashboard & Reports tab - KPI cards and attendance shortage alert

Table 2: Swing components used

Swing component	Variables / use in the project	Purpose
JFrame	MainView extends JFrame; title "Student Management System (Java Swing + SQLite JDBC)", size 1100 x 720, EXIT_ON_CLOSE	Main application window
JTabbedPane, JPanel, JSplitPane	tabbedPane with tabs Students, Courses, Enrollments & Grading, Dashboard & Reports; one JSplitPane per data tab (form on the left, table on the right)	Screen structure and layout
Layout managers	BorderLayout (frame, tabs, table panels), GridBagLayout (forms), GridLayout (button blocks, KPI cards), FlowLayout (search bars)	Arrangement of components
JTextField	txtStudRoll, txtStudName, txtStudEmail, txtStudDept, txtStudPhone, txtStudSearch, txtCourseCode, txtCourseName, txtCourseCredits, txtCourseInstructor, txtCourseSearch, txtAttendancePct, txtExamMarks, txtEnrollSearch	Data entry and live search
JLabel	lblStudPhoneHint, lblCalculatedPreview, lblStatStudents, lblStatCourses, lblStatEnrollments, lblStatAvgScore	Hints, live grade preview and dashboard figures

Swing component	Variables / use in the project	Purpose
JButton	btnStudAdd, btnStudUpdate, btnStudDelete, btnStudClear, btnCourseAdd, btnCourseUpdate, btnCourseDelete, btnCourseClear, btnEnrollSave, btnEnrollDelete, btnEnrollClear, btnRefreshDashboard	User actions
JComboBox<String>	comboEnrollStudent, comboEnrollCourse	Choose the student and course for an enrollment
JTable, DefaultTableModel, TableRowSorter, JScrollPane	studTable, courseTable, enrollTable, shortageTable with studTableModel, courseTableModel, enrollTableModel, shortageTableModel and studSorter, courseSorter, enrollSorter	Display records; selection, sorting and filtering
JOptionPane	MainView.showMessage() and MainView.confirmDialog()	Success, warning and error messages; delete confirmation

Table 3: Event listeners and their handlers

Event source	Listener type	Handler (class and method)
All btn... buttons	ActionListener (lambda e -> ...)	AppController.handleAddStudent(), handleUpdateStudent(), handleDeleteStudent(), clearStudentForm(); the same four for courses; handleSaveEnrollment(), handleDeleteEnrollment(), clearEnrollmentForm(); loadDashboardData() for btnRefreshDashboard
studTable, courseTable, enrollTable (selection model)	ListSelectionListener	Copies the selected row into the form fields (and into the combo boxes and marks fields for enrollments); roll number and course code become read-only
txtAttendancePct, txtExamMarks	DocumentListener (calcListener)	updateLiveGradePreview()
txtStudPhone	DocumentFilter (anonymous subclass) and DocumentListener (phoneListener)	Allows only digits and at most 10 characters; updatePhoneFeedback() updates lblStudPhoneHint
txtStudSearch, txtCourseSearch, txtEnrollSearch	DocumentListener	setupSearchFilter() sets a RowFilter on the TableRowSorter after every keystroke

2.4. Logic & Event Handling

Start-up. Main.main() sets the system look and feel with UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName()) and then, inside SwingUtilities.invokeLater(), creates a DatabaseModel and calls initDatabase(), creates a MainView, and creates an AppController with new AppController(model, view). The AppController constructor calls initListeners() to attach all listeners and loadAllData(), which runs loadStudents(), loadCourses(), loadEnrollments() and loadDashboardData() so that every table and the combo boxes are filled from the database. Finally view.setVisible(true) shows the window. If initDatabase() throws a SQLException, a dialog "Failed to initialize database: ..." is displayed.

Adding a student. The button btnStudAdd triggers AppController.handleAddStudent(). The method reads txtStudRoll, txtStudName, txtStudEmail, txtStudDept and txtStudPhone, trims them, checks that roll number and name are present and that the phone number is present, numeric and exactly 10 digits, converts the roll number with Long.parseLong(), and calls DatabaseModel.addStudent(roll, name, email, dept, phone), which runs an INSERT through a PreparedStatement. On success the view shows a message with showMessage(), then clearStudentForm(), loadStudents() and loadDashboardData() refresh the screen. loadStudents() also calls refreshStudentDropdown() so that the new student appears in comboEnrollStudent.

Selecting and updating a student. When a row of studTable is clicked, the ListSelectionListener converts the row with convertRowIndexToModel(), copies the five values from studTableModel into the text fields and calls txtStudRoll.setEditable(false) so that the primary key cannot be changed. btnStudUpdate calls handleUpdateStudent(), which applies the same validation and then DatabaseModel.updateStudent(), an UPDATE ... WHERE roll_no = ?; the controller then reloads the student and enrollment tables because student names are shown in both. btnStudClear calls clearStudentForm(), which empties the fields, makes txtStudRoll editable again and clears the table selection.

Deleting a student. btnStudDelete calls handleDeleteStudent(). If no roll number is loaded a warning is shown; otherwise MainView.confirmDialog() asks "Deleting this student will also remove their enrollments. Proceed?" and only on Yes is DatabaseModel.deleteStudent(roll) executed, after which the student, enrollment and dashboard data are reloaded.

Courses. The course tab works in the same way through handleAddCourse(), handleUpdateCourse() and handleDeleteCourse(), using DatabaseModel.addCourse(), updateCourse() and deleteCourse(). The course code is converted with toUpperCase() before it is used, and credits are parsed with Integer.parseInt().

Enrollment, marks and grade. handleSaveEnrollment() reads the selected items of comboEnrollStudent and comboEnrollCourse (strings of the form "roll - name" and "code - name", split with split(" - ")[0]), parses txtAttendancePct and txtExamMarks with Double.parseDouble(), checks the ranges 0 to 100 and 0 to 90, and calls DatabaseModel.saveEnrollment(). Inside saveEnrollmentWithConn() the model computes attMarks = calculateAttendanceMarks(attendancePct), total = examMarks + attMarks and grade = calculateGrade(total) and stores them with the INSERT ... ON CONFLICT ... DO UPDATE statement, so saving the same student and course again updates the existing row. The rules used by calculateAttendanceMarks() and calculateGrade() are given in Table 4. For example, attendance 85% and exam marks 90 give 8 + 90 = 98 marks and grade A+ (Figure 5).

Table 4: Attendance marks and grade rules implemented in DatabaseModel

Attendance %	Attendance marks (of 10)	Total marks (of 100)	Grade
90 or more	10	90 or more	A+
85 to below 90	8	80 to below 90	A
80 to below 85	6	70 to below 80	B
75 to below 80	4	60 to below 70	C
Below 75 (shortage)	0	50 to below 60	D
		Below 50	F

Live preview and phone feedback. The DocumentListener calcListener on txtAttendancePct and txtExamMarks calls updateLiveGradePreview() after every keystroke. The method reads both fields, shows a message if they are empty, non-numeric or out of range, and otherwise calls model.calculateAttendanceMarks() and model.calculateGrade() and writes the result to lblCalculatedPreview. Similarly, the DocumentFilter on txtStudPhone rejects any insertion that is not a digit or would make the text longer than 10 characters, and phoneListener calls updatePhoneFeedback(), which changes the text and colour of lblStudPhoneHint (grey for empty, orange while typing, green with "10 digits completed (Limit reached)").

Search. setupSearchFilter(searchField, sorter) adds a DocumentListener to each search box. On every change it calls sorter.setRowFilter(RowFilter.regexFilter("(?i)" + text)), so the table shows only rows in which any column matches the typed text, ignoring case. An empty box removes the filter.

Dashboard. loadDashboardData() calls getStudentCount(), getCourseCount(), getEnrollmentCount() and getAverageTotalMarks() to fill lblStatStudents, lblStatCourses, lblStatEnrollments and lblStatAvgScore, and getLowAttendanceStudents() (a JOIN with WHERE attendance_pct < 75.0 ORDER BY attendance_pct ASC) to fill shortageTableModel. It is called at start-up, after every add, delete or save that can change the counts, and by btnRefreshDashboard.

2.5. Inputs, Outputs and Error handling

Table 5: Inputs, outputs and error handling

Component / Action	Input / Output Type	Validation & Error Handling Rule
Roll Number (txtStudRoll)	Input: long (primary key)	**Check:** must not be empty; must be numeric. **Error:** "Roll Number and Name are required!" for empty; NumberFormatException gives "Roll Number must be a valid number!"; a duplicate roll number raises SQLException and the dialog "Database error: ...". The field is read-only while a table row is selected.
Full Name (txtStudName)	Input: String	**Check:** must not be blank. **Error:** "Roll Number and Name are required!" (add) or "Select a student to update and provide valid fields." (update).
Email (txtStudEmail), Department (txtStudDept)	Input: String (optional)	No validation in the code; the text is stored as typed.
Phone (txtStudPhone)	Input: String of 10 digits	**Check:** DocumentFilter blocks non-digits and more than 10 characters while typing; on Add/Update the phone must be present, digits only and exactly 10 long. **Error:** "Phone number is required and must be a 10-digit number!", "Phone number must contain only numbers (0-9)...", "Phone number must be exactly 10 digits! (You entered n digits)".
Course Code (txtCourseCode)	Input: String (primary key), converted to upper case	**Check:** must not be empty. **Error:** "Course Code, Name, and Credits are required!"; a duplicate code gives "Database error: ...".
Course Name (txtCourseName)	Input: String	**Check:** must not be empty. **Error:** "Course Code, Name, and Credits are required!".
Credits (txtCourseCredits)	Input: int	**Check:** required and must be an integer. **Error:** NumberFormatException gives "Credits must be a valid integer!". The label says 1-5 but the code does not check this range.
Instructor (txtCourseInstructor)	Input: String (optional)	No validation.
Student and Course drop-downs	Input: selected "roll - name" and "code - name"	**Check:** both must have a selection. **Error:** "Please make sure students and courses are available to select."
Attendance % (txtAttendancePct)	Input: double, 0 to 100	**Check:** must be present, numeric and within range. **Error:** "Please enter attendance % and exam marks.", "Attendance and Exam marks must be numeric!", "Attendance percentage must be between 0 and 100."
Exam Marks (txtExamMarks)	Input: double, 0 to 90	**Check:** present, numeric, within range. **Error:** "Exam marks must be between 0 and 90 (10 marks reserved for attendance)."
Delete buttons	Action on selected row	**Check:** a row (or loaded roll number / course code) must exist. **Error:** "Select a student from the table to delete." (similar messages for courses and enrollments). Otherwise a Yes/No confirmation is shown before deletion.
Search boxes	Input: String used as a case-insensitive regular expression	Empty text removes the filter. Invalid regular-expression text (for example a single "(") is not handled by the code and may throw PatternSyntaxException.

Component / Action	Input / Output Type	Validation & Error Handling Rule
Output: tables, preview, dashboard	Output: JTable rows, lblCalculatedPreview text, KPI labels, JOptionPane dialogs	Database read failures show "Failed to load ...: <message>" with the title "Database Error"; successful operations show a "Success" information dialog.

3. Test results

Table 6: Test cases and results

SL	Test case with UI interactions	Expected Result	Actual Result	Status (PASS or FAIL)
i.	Run java -cp ".;sqlite-jdbc-3.53.2.1.jar" Main	Main window opens with tabs Students, Courses, Enrollments & Grading, Dashboard & Reports, and tables filled from the database	Window "Student Management System (Java Swing + SQLite JDBC)" opens with the four tabs and data in the tables (Figures 3 to 6)	PASS
ii.	Students tab: enter roll 13000123031, name, email, department CSE, phone 1523659423 and click 'Add Student'	Record saved; new row appears in the JTable and the SQLite file	Rows 13000123031, 13000124018 and 13000124031 are listed in Figure 3 although the sample data only has 101 to 103, so they were added through the application	PASS
iii.	Type the 10 digits of a phone number	Hint under the field turns green and reads "10 digits completed (Limit reached)"	Hint shown in green; the tick symbol is drawn as an empty box in the screenshot (font lacks the glyph)	PASS
iv.	Click a row in the student table	Fields are filled from the row and the roll number field becomes read-only	Form in Figure 3 shows the selected row's data; roll number field is greyed	PASS
v.	Enrollments tab: choose 13000123031 - Chandrima Saha, CS103, attendance 85, exam 90, click 'Save / Update Grade'	New enrollment row with attendance marks 8.0/10, total 98.0/100, grade A+	Row ID 9 in Figure 5 shows 85.0%, 8.0 / 10, 90.0 / 90, 98.0 / 100, A+	PASS
vi.	Type the same attendance and exam values	Preview line shows attendance marks, total and grade	Preview reads "Attendance Marks: 8.0/10 | Total: 98.0/100 | Grade: ..."; the grade is cut off because the label is wider than the left panel	FAIL (display only)
vii.	Save attendance 50 and exam 82 for student 101 and course CS101 (sample row was 92% and 82)	Existing enrollment updated, no duplicate row; attendance marks 0, total 82, grade A	Row ID 1 in Figure 5 shows 50.0%, 0.0 / 10, 82.0 / 90, 82.0 / 100, A	PASS
viii.	Open Dashboard & Reports	Enrollments with attendance below 75% are listed	Figure 6 lists 101 Alice Johnson CS101 50.0% and 103 Charlie Davis CS103 68.0%	PASS
ix.	Compare dashboard cards with the tables	6 students, 3 courses, 7 enrollments, average total = (98+100+100+52+72+83+82)/7 = 83.9	Figure 6 shows 6, 3, 7 and 83.9	PASS
x.	Click 'Add Student' with the name field empty	Warning "Roll Number and Name are required!"	Not executed yet	
xi.	Enter a 9-digit phone number and click 'Add Student'; try typing letters or an 11th digit	Error "Phone number must be exactly 10 digits! (You entered 9 digits)"; letters and extra digits are ignored	Not executed yet	
xii.	Add a student whose roll number already exists	Dialog "Database error: ..." (primary key violation); no new row	Not executed yet	

SL	Test case with UI interactions	Expected Result	Actual Result	Status (PASS or FAIL)
xiii.	Type "alice" in 'Search Students'	Only rows containing the text (any case) remain	Not executed yet	
xiv.	Delete a student who has enrollments and confirm with Yes	Student and the enrollments are removed; Total Enrollments on the dashboard decreases (tests ON DELETE CASCADE)	Not executed yet	
xv.	Save an enrollment with attendance 120, then exam marks 95	Range-error dialogs for attendance (0-100) and exam marks (0-90)	Not executed yet	
xvi.	Add a record, close the program, run it again	The record is still present (stored in student_management.db)	Not executed yet	

Test cases i to ix were evaluated from screenshots of the running application. Cases x to xvi have not been run yet and should be executed and recorded before submission; case xiv is particularly important because of the note on foreign keys in Section 2.2.

4. Conclusion

The Student Management System is a desktop application for the academic-records domain: it keeps students, courses, enrollments, attendance and marks in one SQLite database and gives staff a four-tab Swing interface to manage them, to see grades computed automatically (90 exam marks plus up to 10 attendance marks) and to spot students with attendance below 75%.

OOP principles. Encapsulation: AppController keeps model and view as private fields and DatabaseModel hides all JDBC and SQL behind public methods such as saveEnrollment() and getAllStudents(); the Swing components of MainView are public, which is a trade-off made so that the controller can attach listeners. Abstraction: the controller works with high-level model operations and never builds SQL itself. Inheritance: MainView extends JFrame, and anonymous subclasses of DefaultTableModel and DocumentFilter customise library classes. Polymorphism: isCellEditable(), insertString() and replace() are overridden, and the same listener interfaces (ActionListener, DocumentListener) are implemented by many different handlers. Composition: AppController has a DatabaseModel and a MainView, and MainView is made of many components. Exceptions: SQLException and NumberFormatException are caught and shown to the user, and try-with-resources closes connections automatically.

MVC pattern. The Model (DatabaseModel) owns data access and the grading rules, the View (MainView) only builds and displays components, and the Controller (AppController) listens to user events, validates input, calls the Model and refreshes the View. Main wires the three together. **GUI.** The interface uses tabs, split panes, forms, tables with live filtering, combo boxes, a live calculation preview and dialogs. **Database connectivity.** JDBC with the sqlite-jdbc driver is used with PreparedStatement, JOIN queries, an upsert and automatic creation of the schema and sample data.

Limitations found while analysing the code and screenshots: (1) foreign-key enforcement is switched on only for the connection used in initDatabase(), so cascade deletion should be verified; (2) the credit range 1 to 5 is shown in the label but not checked; (3) e-mail format is not validated; (4) a search text that is an invalid regular expression is not handled; (5) there is no login or access control, and a new connection is opened for every database call; (6) the live preview and one label are truncated in the Enrollments tab. Fixing these, and adding export of reports and backups, would be natural next steps.

5. References

- Java: A Beginner's Guide, Herbert Schildt, McGraw Hill
- Oracle, "Creating a GUI With Swing," The Java Tutorials. https://docs.oracle.com/javase/tutorial/uiswing/ (accessed 31 Oct 2026)
- Oracle, "JDBC Basics," The Java Tutorials. https://docs.oracle.com/javase/tutorial/jdbc/ (accessed 31 Oct 2026)
- SQLite, "Foreign Key Support." https://www.sqlite.org/foreignkeys.html (accessed 31 Oct 2026)
- Xerial, "sqlite-jdbc" (SQLite JDBC driver). https://github.com/xerial/sqlite-jdbc (accessed 31 Oct 2026)
- Project source code: https://github.com/prateek-kalwar-95/Student-Management-System (accessed 31 Oct 2026)